Практическая работа №2

Аудит готового клиент-серверного приложения

Шаг 1. Ответы на вопросы по коду

Вопрос 1. В send_reliable создаётся сокет SOCK_STREAM и вызывается connect(). Какой это протокол?

Это **TCP**. Тип `SOCK_STREAM` означает потоковый сокет с гарантированной доставкой и сохранением порядка байтов, а именно так работает TCP. Кроме того, вызов `connect()` устанавливает соединение с сервером (трёхстороннее рукопожатие) ещё до отправки данных — это признак TCP, а в самом коде функция названа «надёжной».

Вопрос 2. В send_fast создаётся сокет SOCK_DGRAM и вызывается sendto(). Какой это протокол?

Это **UDP**. Тип `SOCK_DGRAM` — это датаграммный сокет, а `sendto()` отправляет отдельный пакет по адресу без установки соединения. Подтверждения доставки у UDP нет, поэтому клиент задаёт таймаут и ждёт ответ ограниченное время; в коде это `settimeout(2.0)`.

Вопрос 3. В каком формате данные передаются от клиента к серверу?

В формате **JSON**. Подтверждающая строка в `client.py`: `client.send(json.dumps(message, ensure_ascii=False).encode("utf-8"))` — словарь превращается в JSON-строку через `json.dumps`. Сервер в ответ делает обратное: `json.loads(data.decode("utf-8"))`.

Вопрос 4. Что будет, если клиент отправит обычную строку «hello»?

Вызов `json.loads` не сможет разобрать текст и выбросит `JSONDecodeError`. Исключение перехватывается блоком `except`: сервер печатает в консоль «Ошибка: не удалось разобрать JSON» и отправляет клиенту ответ `{"status": "ERROR", "reason": "invalid JSON"}`. Сервер при этом не падает и продолжает работать (проверено на практике, см. случай 3.3).

Шаг 2. Запуск сервера и клиента

Сервер запущен в одном окне терминала (`python server.py`), клиент — в другом (`python client.py`).

```
[TCP] Сервер запущен на localhost:9000
[UDP] Сервер запущен на localhost:9001
[TCP] Получено от ('127.0.0.1', 34284): {'city': 'Москва', 'temp': -5, 'humidity': 80,
'is_snowing': True}
[UDP] Получено от ('127.0.0.1', 34093): {'city': 'Москва', 'temp': -5, 'humidity': 80,
'is_snowing': True}
```

Окно 1 — сервер: после запуска и после приёма двух сообщений от клиента.

```
=== Отправка через надёжный протокол ===
Ответ: {'status': 'OK', 'protocol': 'TCP', 'format': 'JSON'}
Время: 0.16 мс

=== Отправка через быстрый протокол ===
Ответ: {'status': 'OK', 'protocol': 'UDP', 'format': 'JSON'}
Время: 0.14 мс
```

Что видно: сервер поднял два сокета (TCP на порту 9000, UDP на порту 9001) и получил от клиента один и тот же JSON-объект с погодой по обоим протоколам. Клиент получил ответ `status: OK` и от TCP, и от UDP, при этом оба обмена заняли доли миллисекунды, так как всё происходит на локальной машине.

Шаг 3. Проверка граничных случаев

Случай 3.1. Сервер выключен, клиент TCP

```
=== Отправка через надёжный протокол ===
Traceback (most recent call last):
  File "/home/claude/work/client.py", line 45, in <module>
    response, elapsed = send_reliable(weather)
                        ^^^^^^^^^^^^^^^^^^^^^^
  File "/home/claude/work/client.py", line 20, in send_reliable
    client.connect((HOST, PORT_R))
ConnectionRefusedError: [Errno 111] Connection refused
```

Что произошло: клиент сразу завершился с ошибкой `ConnectionRefusedError: [Errno 111] Connection refused` (в Windows — `WinError 10061`). **Почему:** TCP требует установить соединение, а на порту 9000 никто не слушает, поэтому операционная система отвечает отказом. Ошибка возникает в `client.connect()`; в коде нет `try/except`, поэтому программа аварийно завершается, и до UDP-части дело даже не доходит.

Случай 3.2. Сервер выключен, клиент UDP

```
Ответ: {'status': 'TIMEOUT'}
Время: 2002.30 мс
```

Что произошло: клиент ничего не получил и спустя примерно **2 секунды** (ровно столько задано в `settimeout(2.0)`) вернул `{'status': 'TIMEOUT'}`. Ошибки при этом нет. **Почему:** UDP не устанавливает соединение и не проверяет доставку, поэтому `sendto()` «успешно» отправляет датаграмму в никуда, а узнать об отсутствии получателя клиент может только по истечении таймаута. Это принципиальное отличие от TCP, который сообщает об ошибке сразу. (В Windows в такой ситуации иногда вместо таймаута возникает `ConnectionResetError`, который в коде тоже не обрабатывается.)

Случай 3.3. Клиент отправляет не JSON

В `client.py` строка отправки временно заменена на `client.send(b"hello")`, после проверки возвращена обратно.

```
=== Отправка через надёжный протокол ===
Ответ: {'status': 'ERROR', 'reason': 'invalid JSON'}
Время: 0.17 мс

=== Отправка через быстрый протокол ===
Ответ: {'status': 'OK', 'protocol': 'UDP', 'format': 'JSON'}
Время: 0.11 мс
```

```
[TCP] Ошибка: не удалось разобрать JSON от ('127.0.0.1', 34298)
```

Что произошло: сервер ответил `{'status': 'ERROR', 'reason': 'invalid JSON'}`, а в его консоли появилась строка «[TCP] Ошибка: не удалось разобрать JSON». **Почему:** «hello»

— не валидный JSON, `json.loads` выбросил `JSONDecodeError`, который обработан в блоке `except`. UDP-часть клиента (она отправляла корректный JSON) отработала нормально. Дополнительно проверено: тот же «hello», отправленный на UDP-порт, даёт такой же ответ ERROR.

Случай 3.4. Сравнение форматов

```
JSON: {"city": "Москва", "temp": -5, "humidity": 80, "is_snowing": true}
Размер JSON: 72 байт
XML:
<weather><city>Москва</city><temp>-5</temp><humidity>80</humidity><is_snowing>True</
is_snowing></weather>
Размер XML: 111 байт

Восстановлено из JSON: {'city': 'Москва', 'temp': -5, 'humidity': 80, 'is_snowing':
True}
Восстановлено из XML: {'city': 'Москва', 'temp': '-5', 'humidity': '80', 'is_snowing':
'True'}
Данные совпадают: True
```

Формат	Размер	Строка
JSON	72 байта	{"city": "Москва", ...}
XML	111 байт	<weather><city>Москва</city>...</weather>

Какой формат компактнее? JSON: 72 байта против 111 байт у XML (на 54% больше). **Почему:** в XML каждое имя поля записывается дважды — в открывающем и закрывающем теге (`<temp>-5</temp>`), а в JSON — один раз в виде ключа, плюс XML добавляет корневой тег `<weather>`.

Замечание аудитора: строка «Данные совпадают: True» вводит в заблуждение — сравнивается только `restored_json == weather`, то есть JSON с самим собой, а XML не проверяется. На деле из XML все значения восстанавливаются как строки (`'-5'`, `'80'`, `'True'`), и `restored_xml == weather` дало бы `False`: XML не хранит типы данных, а JSON сохраняет числа и булевы значения.

Дополнительные замечания по результатам аудита

При чтении кода и проверках найдены следующие проблемы (первая проверена запуском):

- **Сервер падает на не-UTF-8 данных.** В `server.py` перехватывается только `JSONDecodeError`, а `UnicodeDecodeError` (например, байты `\xff\xfe`) — нет. При проверке TCP-поток сервера аварийно завершился, и следующее подключение к порту 9000 дало «Connection refused», тогда как UDP продолжил работать. В UDP-цикле такая же ошибка остановила бы весь сервер.
- **Нет обработки ошибок в клиенте.** При недоступном сервере TCP-клиент падает с traceback (случай 3.1), а ответ сервера не проверяется на валидность.
- **TCP-сервер обрабатывает клиентов строго по одному.** Пока один клиент не отправил данные, остальные ждут: у `conn.recv()` нет таймаута.

- **Одиночный recv(4096) без разделителя сообщений.** TCP — поток байтов, поэтому большое сообщение (> 4096 байт) может прийти частями и будет прочитано неполным.
- **Неравное сравнение времени.** В `send_reliable` замер начинается после `connect()`, поэтому время установки TCP-соединения не учитывается, и сравнение TCP с UDP по скорости некорректно.
- **Мелочи:** в `server.py` импортирован `xml.etree`, но не используется; в тексте задания ожидаемый вывод содержит заголовки «TCP/UDP», а в коде — «надёжный/быстрый протокол»; у UDP нет повторной отправки при потере пакета.

Вывод

Протокол. Для передачи данных о погоде я бы выбрал **TCP**: ответ сервера нельзя потерять молча, а при недоступности сервера TCP сразу сообщает об ошибке (случай 3.1), тогда как UDP заставляет ждать таймаут и не гарантирует доставку. Выигрыш UDP в скорости на практике ничтожен (доли миллисекунды). UDP был бы уместен только для частых некритичных обновлений, где потеря одного пакета не страшна.

Формат. Я бы выбрал **JSON**: он компактнее (72 против 111 байт), проще читается, сохраняет типы данных (числа, булевы значения) и естественно сопоставляется со словарями Python. XML имеет смысл, когда нужны схема валидации и атрибуты, но для простого объекта «погода» он избыточен.

Итог аудита: приложение в штатном режиме работает корректно, но не готово к использованию «как есть» из-за слабой обработки ошибок (особенно падения сервера на некорректной кодировке) и отсутствия таймаутов и обработки исключений в клиенте.